

GIT · IA · DOCS

CAPACITAÇÃO INTERNA

GIT, GITLAB E IA APLICADA À ENGENHARIA DE SOFTWARE

Versionamento, colaboração e documentação de sistemas legados a partir de dois problemas reais

Ronivaldo D. Andrade

Estagiário de Análise e Desenvolvimento de Sistemas
Marinha do Brasil

01/10/2026 · 10h às 11h



01

Fundamentos

Git, GitLab, branch, Merge Request, glab

02

Problema 1

Trazer um sistema já versionado para o GitLab institucional

03

Problema 2

Documentar sistemas legados com um agente de IA

04

Encerramento

A visão geral e o que levar daqui

Ao final, você deve conseguir:

- diferenciar **Git**, **GitLab**, **GitHub** e **glab**;
- explicar branch, commit, merge e Merge Request;
- entender e resolver um conflito;
- unir dois históricos Git independentes;
- usar um agente de IA como ferramenta de engenharia, com validação humana.



Os exercícios práticos estão na **apostila**. Aqui é só a exposição.

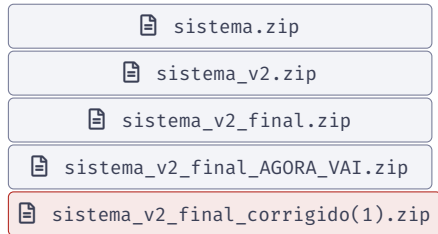
01

FUNDAMENTOS

Git, GitLab, branches, Merge Requests e o GitLab CLI

GIT · IA · DOCS

Sem controle de versão, o histórico vira isto:



❌ Quem mudou? O quê? Por quê? Qual é a certa?

Com o Git, cada mudança fica registrada com:

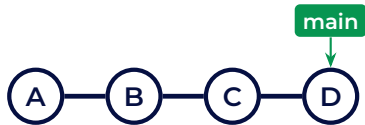
- **o que** mudou (diferença linha a linha);
- **quem** mudou e **quando**;
- **por que** mudou (mensagem do commit);
- a possibilidade de **voltar** a qualquer estado anterior;
- várias pessoas trabalhando **em paralelo**.



Um **sistema distribuído de controle de versão**.

- Cada **commit** é uma fotografia do projeto num momento.
- Os commits formam uma **linha do tempo**: o histórico.
- Funciona **100 % local**, sem servidor nenhum:

```
git init
git add .
git commit -m "Primeiro commit"
```



cada círculo = um commit
(quem, quando, o quê, por quê)



“Isto é o Git. Ele controla trabalho colaborativo em projetos por meio de um belo modelo distribuído de árvore da teoria dos grafos.”

“Legal. Como a gente usa?”

“Não faço ideia. Só decora estes comandos... se der erro, salva teu trabalho em outro lugar, apaga o projeto e baixa uma cópia nova.”

Hoje a proposta é o contrário: **entender o modelo** por trás dos comandos, para não precisar apagar nada.

xkcd #1597 “Git”, de Randall Munroe — CC BY-NC 2.5 — tradução livre.

GIT, GITLAB, GITHUB E GLAB NÃO SÃO A MESMA COISA

Git · IA · Docs

	O que é	Para que serve
 git	Git	Sistema distribuído de controle de versão
	GitLab	Plataforma baseada em Git, instalável na própria infraestrutura
	GitHub	Plataforma baseada em Git, majoritariamente em nuvem
	GitLab CLI glab	Ferramenta de linha de comando

Git é o motor. **GitLab** e **GitHub** são a garagem com oficina, vistoria e controle de acesso. **glab** é o controle remoto da garagem.

GITLAB: UMA PLATAFORMA INTEIRA EM VOLTA DO GIT

· IA · DOCS



Repositórios



Branches



Merge Requests



Revisão de código



Permissões



Issues



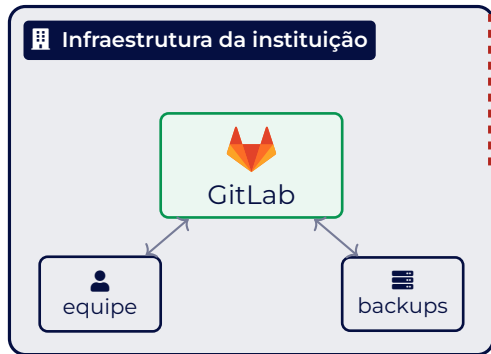
CI



Releases



Auditoria, segurança e colaboração entre equipes

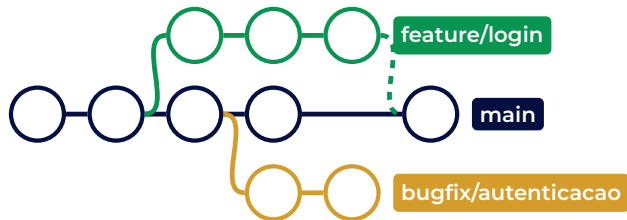


internet
externa

A instituição instala e administra a **própria instância**. Ganha controle sobre:

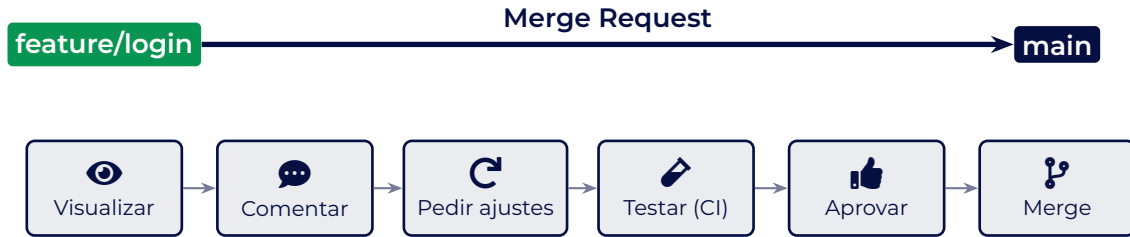
- código-fonte e histórico;
- usuários, permissões e autenticação;
- auditoria e políticas de segurança.

⚠ Estar dentro de casa não é estar seguro. Segurança continua dependendo de configuração, atualização, backup, firewall, monitoramento e controle de acesso.



- Trabalho **isolado**, sem mexer na main.
- Várias pessoas **em paralelo**.
- Espaço para **experimental** e corrigir.
- Revisão **antes** de integrar.

Branch **não** é uma cópia do projeto: é um **ponteiro** para uma linha do histórico.



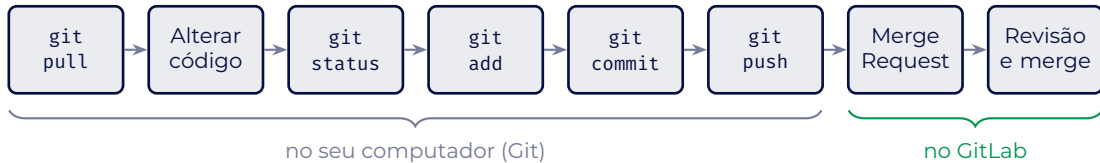
Commit registra uma alteração > **Branch** cria uma linha de trabalho > **MR** pede a integração

Começar e registrar

<code>git clone</code>	copiar um repositório remoto
<code>git init</code>	criar um repositório local
<code>git status</code>	ver o estado atual
<code>git add</code>	preparar alterações
<code>git commit</code>	registrar no histórico
<code>git log</code>	consultar o histórico
<code>git diff</code>	comparar alterações
<code>git restore</code>	desfazer mudanças num arquivo

Branches e remoto

<code>git branch</code>	listar ou criar branches
<code>git switch</code>	trocar de branch
<code>git merge</code>	unir históricos
<code>git remote</code>	gerenciar remotos
<code>git fetch</code>	baixar do remoto, sem mesclar
<code>git pull</code>	baixar e mesclar
<code>git push</code>	enviar para o remoto

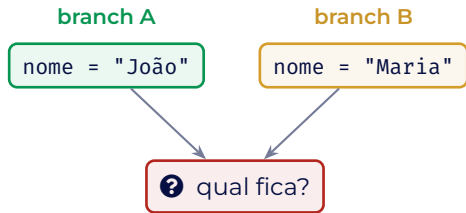


💡 Regra de ouro: **comece sempre com `git pull`** e trabalhe numa branch própria. Quanto mais tempo longe da main, maior a chance de conflito.

CONFLITO: QUANDO O GIT PEDE UMA DECISÃO HUMANA

IT · IA · DOCS

Duas branches mudaram **a mesma linha**:



Conflito **não é erro**. É o Git dizendo: “não sei combinar isso sozinho, decida você”.

O Git marca o arquivo assim:

```
<<<<<< HEAD
nome = "João"
=====
nome = "Maria"
>>>>>> branch-b
```

1. edite e deixe só o que deve ficar;
2. apague os marcadores <<<, ==, >>>;
3. git add arquivo e git commit.

```
# autenticar (uma vez)
```

```
glab auth login --hostname  
    <endereco-do-gitlab>
```

```
# criar um Merge Request da branch atual
```

```
glab mr create --target-branch main \  
    --title "Integra o sistema"
```

```
# acompanhar
```

```
glab mr list
```

```
glab mr view
```

```
glab ci status
```

Git controle de versão

GitLab plataforma de colaboração

glab o GitLab no terminal

As **regras de permissão** do GitLab valem também no terminal: se a main exige aprovação, o glab não pula essa etapa.

02

PROBLEMA 1

Um sistema já versionado localmente precisa entrar no GitLab institucional, onde a `main` já existe e está protegida

GIT · IA · DOCS

Ao criar um repositório, o GitLab institucional **já entrega**:

- uma branch `main` criada;
- um `README.md` padrão;
- a `main` **protegida**: nada de push direto;
- mudanças só por **Merge Request**.

Mas o sistema **já existia** no computador, com anos de histórico Git próprio.



❌ `git push origin main` recusado

O PROBLEMA: DOIS HISTÓRICOS QUE NÃO SE CONHECEM

REPOSITÓRIO LOCAL (anos de trabalho)



REPOSITÓRIO GITLAB (recém-criado)

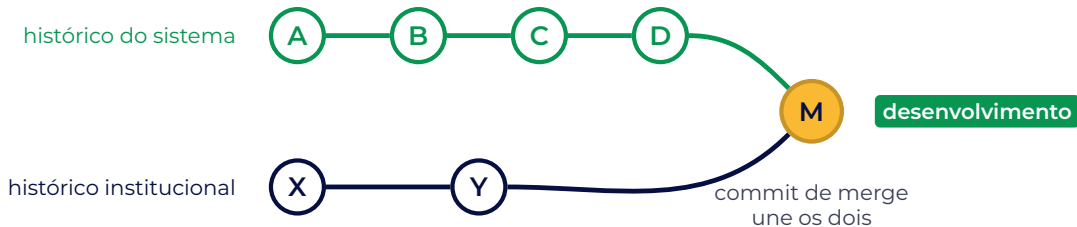


nenhum ancestral
em comum

Um git merge comum
simplesmente recusa:

```
$ git merge origin/main  
fatal: refusing to merge  
unrelated histories
```

O Git só sabe mesclar linhas que **nasceram de um mesmo ponto**. Aqui, não há esse ponto.



Depois do commit **M**, os dois históricos passam a ter ancestral comum. A partir daí um Merge Request para a main flui **sem conflito**.

1. dar ao Git local um endereço remoto

```
git remote add origin  
    <URL_DO_REPOSITORIO>  
git remote -v
```

2. baixar o que existe no GitLab
(sem mesclar nada ainda)

```
git fetch origin
```

3. tirar a main local do caminho

```
git branch -m main desenvolvimento
```

desenvolvimento

seu histórico (A–D),
com novo nome

origin/main

cópia de leitura da
main institucional

Renomear evita confundir a
sua main com a main **prote-
gida** do GitLab.

```
git merge --allow-unrelated-histories origin/main
```

✗ **NÃO significa**

“ignore todos os conflitos”.

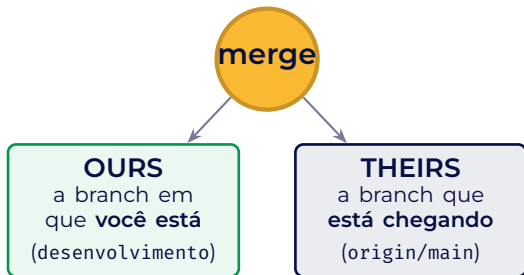
✓ **Significa**

“eu sei que não há ancestral comum e autorizo o Git a tentar juntar”.

Resultado típico: os dois lados têm um README.md diferente.

```
CONFLITO (adicionar/adicionar): conflito de mesclagem em README.md
Automatic merge failed; fix conflicts and then commit the result.
```

PASSO 5: OURS E THEIRS – O PONTO QUE MAIS CAUSA ERRO



manter o README do sistema

```
git restore --ours README.md
```

forma antiga equivalente

```
git checkout --ours README.md
```

manter o README institucional

```
git restore --theirs README.md
```

⚠ ours **não** quer dizer “o arquivo do meu computador”. Quer dizer **o lado da branch atual naquele merge**. Confunda isso e você preserva o arquivo errado.

6. marcar o conflito como resolvido

```
git add README.md
```

```
git commit # grava o M
```

7. enviar a nova branch ao GitLab

```
git push origin desenvolvimento
```

No GitLab passam a existir, lado a lado:



intocada e protegida

desenvolvimento

sistema + histórico integrado

desenvolvimento

Merge Request
revisão + aprovação

 main

- Feito na **interface web** do GitLab, com aprovação de quem tem permissão.
- Ou pelo terminal, com o glab:

```
glab mr create -s desenvolvimento  
-b main
```

Como os históricos agora são relacionados, o merge **não é recusado**: todos os conflitos já foram tratados localmente.

git remote add origin <URL_DO_REPOSITORIO>	# 1. conectar
git fetch origin	# 2. baixar
git branch -m main desenvolvimento	# 3. renomear
git merge --allow-unrelated-histories origin/main	# 4. unir
git restore --ours README.md	# 5. decidir o conflito
git add README.md && git commit	# 6. registrar
git push origin desenvolvimento	# 7. publicar
glab mr create -s desenvolvimento -b main	# 8. pedir a integração

 A apostila traz este mesmo caso para você **reproduzir no seu computador**, sem precisar de servidor.

03

PROBLEMA 2

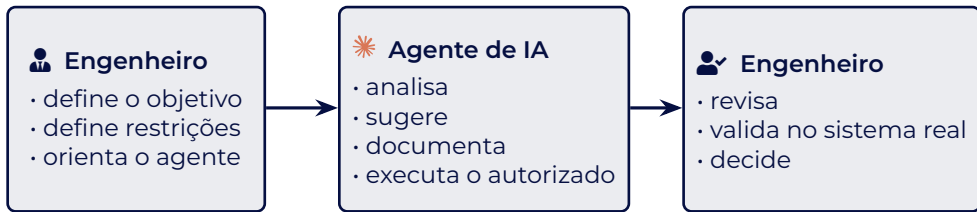
Os sistemas agora estão versionados. Mas quase nenhum tinha documentação

GIT · IA · DOCS

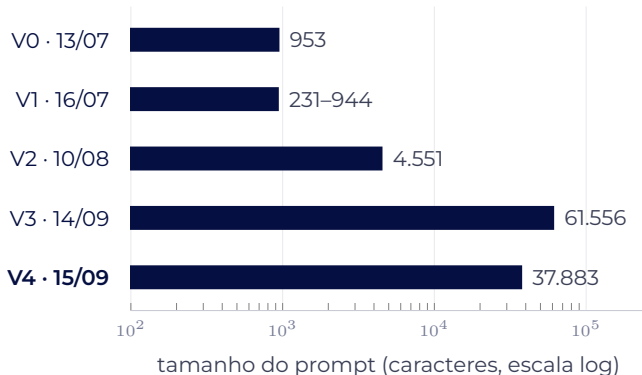


- documentação ausente ou desatualizada;
- regras de negócio só na cabeça de alguém;
- dependências e configurações desconhecidas.

A solução adotada: um **agente de IA** (Claude Code) para acelerar o entendimento, a refatoração e a documentação profunda.



A IA pode interpretar errado regras de negócio, intenções e comportamentos implícitos. **Toda documentação gerada por IA é validada contra o sistema real.** A decisão final é sempre humana.



V0 pipeline em PT: “não modifique”, descubra antes.

V1 pedidos curtos em PT, crescendo a cada reuso.

V2 lista estruturada em PT: 47 itens.

V3 protocolo em inglês, fase *read-only*.

V4 prompt final em inglês, 42 seções, **saída obrigatória em PT-BR**.

Fonte: histórico de prompts do Claude Code do autor, 10/07 a 28/09/2026 — 717 prompts em 123 sessões. V1 mostra a faixa observada; o 1º pedido livre tinha 231 caracteres.

O que deu errado

A regra que entrou no prompt

Documentação superficial, pedida de novo

→ **Análise em fases, arquivo a arquivo**

Agente preenchia lacunas sozinho

→ **Registrar como GAP; nunca inventar**

Risco de alterar código na análise

→ **Fase 0 somente leitura (*read-only*)**

Prompt em inglês, resposta em inglês

→ **Exigência global: tudo em PT-BR**

Agente seguia em frente diante de risco

→ **Protocolo *Safe Stop-and-Report***

~53%

das tarefas de documentação precisaram de **retrabalho** com prompts livres (V1, V2)
8 de 15 tarefas



~25%

com prompts estruturados com contrato de execução (V0, V3, V4)
1 de 4 tarefas

Leia com cautela: estimativa com amostra pequena. “Retrabalho” = refazer, reenviar, “continue de onde parou”, idioma errado. Parte das paradas veio de limite de uso, não do prompt.

164×

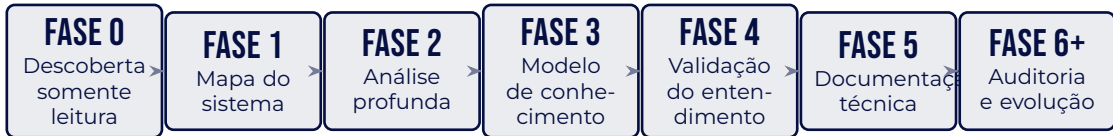
crescimento do prompt, do 1º pedido livre ao final

5 regras

nasceram de 5 falhas observadas

0

pedidos para refazer a doc após o V4 (1 uso)



Evita a cascata: *entendimento errado* → *documentação errada* → *recomendação errada* → *projeto futuro errado*.

Em vez disso: **descobrir** → **evidenciar** → **modelar** → **validar** → **documentar** → **auditar** → **evoluir**.



READ-ONLY DISCOVERY

Na descoberta, o agente **não** altera código, não apaga, não renomeia, não mexe em configuração nem faz commit.

Descobrir antes de modificar.



NUNCA INVENTAR

Todo fato precisa de evidência. O que não se confirma recebe rótulo:

DESCONHECIDO · NÃO
VERIFICADO · INFERIDO ·
REQUER VALIDAÇÃO

E separa **AS-IS**, **TO-BE** e
GAP.

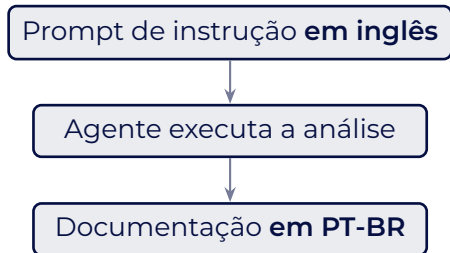


SAFE STOP-AND-REPORT

Diante de dúvida, contradição, risco ou falta de autorização:

**PARAR → REPORTAR →
AGUARDAR** a decisão
humana.

Gravidade de STOP-P0
(crítico) a STOP-P3.



- Idioma do prompt $\neq$ idioma do resultado.
- O V3, só em inglês, fez o agente **responder em inglês**.
- No dia seguinte, o V4 ganhou a regra:

ALL CONTENT YOU PRODUCE MUST BE
WRITTEN IN BRAZILIAN PORTUGUESE.

A economia de tokens com o inglês é a
motivação adotada; ela não foi medida no
histórico.

Seis perguntas que guiam o agente

Descoberta

O que existe neste sistema?

Compreensão

Como as partes se relacionam?

Funcionamento

Como o sistema funciona?

Auditoria

Quais os riscos e pontos fracos?

Documentação

Como virar conhecimento útil?

Validação

Isso bate com o sistema real?

De “código que poucas pessoas conhecem” para “sistema que outras pessoas entendem e mantêm”.

Exemplo do que sai em docs/

docs/

- 00-discovery/DISCOVERY.md
- 00-discovery/SYSTEM_MAP.md
- architecture.md
- business-rules.md
- database.md · api.md
- security.md
- known-issues.md
- technical-debt.md
- guia para leigos

README.md (raiz, profissional)

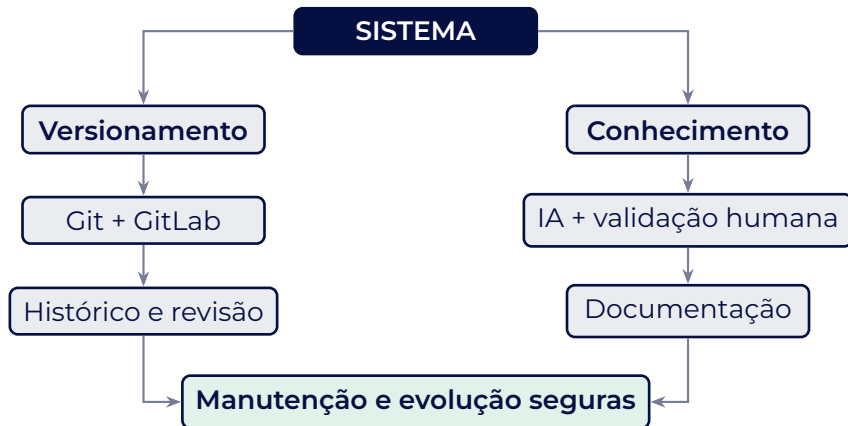
A estrutura exata se adapta a cada sistema.

04

ENCERRAMENTO

A visão geral e a mensagem final

GIT · IA · DOCS



O conhecimento técnico da equipe **não pode morar só na cabeça** de quem escreveu o sistema.

- O código tem **histórico**.
- O sistema tem **documentação**.
- As alterações têm **revisão**.
- As decisões têm **contexto**.

A IA acelera esse trabalho **dentro de um processo controlado**: objetivo claro, restrições bem definidas e validação humana.

**TORNAR OS SISTEMAS MAIS FÁCEIS
DE COMPREENDER, MANTER,
COMPARTILHAR E EVOLUIR.**

OBRIGADO!

Perguntas?

 A **apostila** traz tudo o que vimos com mais calma, o passo a passo para reproduzir o Problema 1 e os **exercícios práticos**.

GIT · IA · DOCS